bezierkit: Bézier Curve Output Tools

Pin Yue Sung[*†] Ling Tak Douglas Chung[*]

2026-10-22 · v1.1.0 · ·

Contents

1 Introduction 2
1.1 Notation 2
1.2 Mathematics and proofs 2
1.3 Reading guide 2
2 Installation 3
2.1 Requirements 3
2.2 Installing the package 3
2.3 Development setup 3
3 Quick start 3
3.1 A cubic curve 3
3.2 The pieces 4
4 Geometry values 4
4.1 Points and vectors 4
4.2 Point sets 5
4.3 Parameters 5
4.4 Errors 5
5 Bézier curves 6
5.1 The Bernstein basis 6
5.2 Curves 7
5.3 Evaluation 8
5.4 Degree elevation 8
6 Derivatives, reversal and subdivision 9
6.1 Derivatives 9
6.2 Reversal 10
6.3 Subdivision 10
7 Cubic segments and paths 11
7.1 Cubic segments 11
7.2 Tight bounding boxes 11
7.3 Degree elevation 12
7.4 Piecewise paths 13
7.5 Continuity 13
8 Constructions and Hermite interpolation 14
8.1 Endpoint constructions 14
8.2 Hermite interpolation 15
8.3 Interpolation error 16
9 Fitting 17
9.1 Adaptive Hermite fitting 17
9.2 Polyline simplification 18
10 Level sets 19
10.1 Level sets and their tangents 19
10.2 Tracing 20
10.3 Results 22
10.4 Limitations 22
11 Sampling and export 22
11.1 Sampling 22
11.2 Rounding and transforms 23
11.3 JSON 23
11.4 SVG 24
11.5 TikZ 24
11.6 Matplotlib 24
12 Command-line interface 24
12.1 Control points 25
12.2 Commands 25
12.3 Errors 25
A Proofs 25
A.1 The Bernstein basis and evaluation 25
A.2 Derivatives, reversal and subdivision 27
A.3 Cubic segments and paths 28
A.4 Constructions and interpolation 30
A.5 Fitting and level sets 31
A.6 Export 32
Change history 33
Command Index 34
References 35

[*]Department of International Business, National Chengchi University, Taipei, Taiwan.
[†]Corresponding author. Email: contact@econ-viz.org

1 Introduction

The bezierkit package is a small mathematical toolkit for Bézier curves. It builds curves from control points or from endpoint conditions, evaluates and differentiates them, splits and restricts them, fits them to functions, sampled points and level sets, and writes them out as JSON, SVG path data or TikZ. It does not plot. Renderers such as Matplotlib, mosaickit or a LaTeX document receive exact cubic control points and draw them.

1.1 Notation

A point or vector lives in $\mathbb{R}^d$ for some dimension $d \geq 1$; most figures use $d = 2$. The Euclidean norm of $x \in \mathbb{R}^d$ is

$$\|x\| = \sqrt{x_1^2 + \ldots + x_d^2},$$

and $\|x\|_\infty = \max_k |x_k|$ is the maximum norm. A set $S \subseteq \mathbb{R}^d$ is convex if $\lambda p + (1-\lambda)q \in S$ whenever $p, q \in S$ and $\lambda \in [0,1]$; the convex hull of a finite set of points is the set of all their convex combinations $\sum_i \lambda_i P_i$ with $\lambda_i \geq 0$ and $\sum_i \lambda_i = 1$. A map $A : \mathbb{R}^d \to \mathbb{R}^e$ is affine if $A(x) = Mx + v$ for a matrix M and a vector v. A function is C^k if it has continuous derivatives up to order k.

A Bézier curve of degree n has $n+1$ control points $P_0, \ldots, P_n$ and is the map

$$B(t) = \sum_{i=0}^{n} b_{i,n}(t) P_i, \quad t \in [0,1],$$

where

$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

are the Bernstein polynomials (Section 5). The control points, joined in order, form the control polygon. Every curve in the package is parameterized over $[0,1]$; a parameter outside it raises `ParameterOutOfDomain`. The letter d always denotes the dimension; tolerances are written ε.

1.2 Mathematics and proofs

Each chapter first recalls the standard definitions it uses, with a reference, and then states the properties the algorithms rely on as numbered lemmas, propositions, theorems and corollaries: the Bernstein basis, evaluation and subdivision by de Casteljau's algorithm, the Hermite error bound and the rounding error of the exporters. Their proofs are collected in Appendix A, so the chapters can be read for the API alone. Conventions that belong to this package, and not to the literature, are called package conventions. The standard references are Farin (2002), Prautzsch et al. (2002) and, for the Bernstein basis, Farouki (2012).

1.3 Reading guide

Table 1 Chapter guide

Topic	Contents	Section
Foundations	Points, vectors, parameters, errors	Section 4
	Bernstein basis, curves, evaluation	Section 5
	Derivatives, reversal, subdivision	Section 6
Cubics	Cubic segments and piecewise paths	Section 7
	Constructions and Hermite interpolation	Section 8

Topic	Contents	Section
Approximation	Fitting functions and polylines	Section 9
	Tracing level sets	Section 10
Output	Sampling, JSON, SVG, TikZ, Matplotlib	Section 11
	Command line	Section 12

On first use, read Section 3 and Section 5. The figures in this manual are themselves bezierkit output: every curve in them was written by the TikZ exporter of Section 11 and compiled with LaTeX.

The scripts that draw the figures of this manual come as one zip, sorted by topic: bezierkit-examples.zip.

2 Installation

2.1 Requirements

bezierkit requires Python 3.10 or later and NumPy. Two optional extras add the rest:

`cli` The `bezierkit` command (Section 12); pulls in Typer and Rich.

`matplotlib` Conversion to and from Matplotlib `Path` objects (Section 11.6).

2.2 Installing the package

```
uv add bezierkit                 # the library
uv add "bezierkit[cli]"          # with the command-line interface
uv add "bezierkit[matplotlib]"   # with the Matplotlib adapter
uv add "bezierkit==1.1.0"        # the version this manual describes
```

With pip, use `python -m pip install bezierkit` and the same extras. Version 1.0.0 is the first stable release: the names exported from `bezierkit` and its documented subpackages follow semantic versioning, and the JSON path schema stays at version 1.

2.3 Development setup

```
git clone https://github.com/EconViz/bezierkit.git
cd bezierkit
uv sync --all-extras --dev
uv run pytest
uv run ruff check src tests benchmarks
uv run lint-imports
```

The continuous-integration workflow runs the same commands on Python 3.10, 3.11, 3.12 and 3.13. `lint-imports` checks the package layering: the mathematical core imports no renderer, and only the adapters import Matplotlib.

3 Quick start

3.1 A cubic curve

The curve used throughout this manual is the cubic with control points $(0,0)$, $(1,2)$, $(3,2)$ and $(4,0)$ (Figure 1).

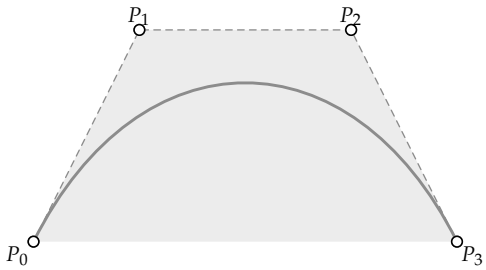


Figure 1
Cubic with control polygon and hull

Example 1

```
from bezierkit import BezierCurve, PiecewiseBezier, Point
from bezierkit.bezier import to_cubic
from bezierkit.export.tikz import to_tikz

curve = BezierCurve.cubic(Point(0, 0), Point(1, 2), Point(3, 2), Point(4, 0))
print(curve.at(0.5))                # Point(coords=(2.0, 1.5))
print(curve.derivative().at(0.5))   # Point(coords=(4.5, 0.0))
left, right = curve.split(0.4)      # two cubics that together trace curve

path = PiecewiseBezier([to_cubic(curve)])
print(to_tikz(path, precision=2, options="thick"))
# \draw[thick] (0.00,0.00) .. controls (1.00,2.00) and (3.00,2.00) .. (4.00,0.00);
```

The curve starts at P_0 and ends at P_3, leaves P_0 towards P_1 and arrives at P_3 from P_2, and never leaves the shaded hull of its control points (Corollary 7). The last line is the curve as a TikZ path, ready to paste into a LaTeX document.

3.2 The pieces

- Values: `Point`, `Vector` and `PointSet` are immutable and dimension-checked (Section 4).
- Curves: `BezierCurve` has any degree and dimension; `CubicBezierSegment` is the cubic that renderers consume; a `PiecewiseBezier` joins cubics into a path (Section 5, Section 7).
- Constructions: endpoint conditions, slopes, derivatives or Hermite data become control points (Section 8).
- Approximation: smooth functions, sampled points and level sets $F(x, y) = c$ become cubic paths with a stated error (Section 9, Section 10).
- Output: samples, JSON, SVG path data and TikZ, and Matplotlib paths (Section 11).

4 Geometry values

All values are immutable: operations return new objects. Coordinates must be finite; NaN and infinities raise `ValueError` when a value is created.

4.1 Points and vectors

`Point` `Point(⟨float⟩, ...)`
`Vector` `Vector(⟨float⟩, ...)`

A point or a vector in $\mathbb{R}^d$, given by its $d \geq 1$ coordinates. Points and vectors combine as in affine geometry (Table 2).

Table 2
Arithmetic of points and vectors

Expression	Result
`point - point`	A `Vector`

Expression	Result
`point + vector`, `point - vector`	A `Point`
`vector + vector`, `vector * scalar`	A `Vector`; `scalar * vector` works too

`coords` The coordinate tuple.

`dimension` The dimension d.

`x` The first coordinate, when it exists.

`y` The second coordinate, when it exists.

`z` The third coordinate, when it exists.

`dot()` The inner product of two vectors.

`norm()` The Euclidean length of a vector.

`normalized()` The unit vector in the direction of a vector; a zero vector cannot be normalized.

Mixing dimensions raises `DimensionMismatch`. `point + point` is not rejected: it adds the coordinates and returns a `Point`, which is meaningful only in combinations whose weights sum to one, such as the averages inside de Casteljau's algorithm.

```
from bezierkit import Point, Vector

p = Point(1, 2)
q = p + Vector(3, -1) * 2      # Point(coords=(7.0, 0.0))
v = q - p                      # Vector(coords=(6.0, -2.0))
print(v.norm())                # 6.324555320336759
```

4.2 Point sets

`PointSet` `PointSet(`⟨*points*⟩`)`

An immutable batch of points backed by a read-only (count, d) NumPy array, as returned by batch evaluation and sampling. It offers `count`, `dimension`, `array` (a read-only copy), the columns `x`, `y`, `z`, iteration over `Point`s and indexing.

4.3 Parameters

`Interval` `Interval(start, end)` is a closed interval with `contains()`, `clamp()` and `linspace()`.

`ParameterValues` Validates a scalar or a one-dimensional array of parameters against a domain. Every curve uses it, so `at(1.2)` on a curve over $[0, 1]$ raises `ParameterOutOfDomain`.

4.4 Errors

Table 3 Exceptions

Exception	Raised when
`BezierKitError`	Base class of the exceptions below
`DimensionMismatch`	Points, vectors or segments of different dimensions are combined
`DegreeError`	A curve has the wrong degree for an operation, or no control points
`ParameterOutOfDomain`	A parameter lies outside the curve's domain $[0, 1]$
`ToleranceNotMet`	Adaptive fitting cannot reach its tolerance within its limits (Section 9)

Invalid arguments that are not geometry, such as a negative tolerance, raise `ValueError`.

5 Bézier curves

5.1 The Bernstein basis

Definition 1 (Bernstein polynomials). The Bernstein polynomials of degree $n \geq 0$ are

$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad i = 0, \ldots, n.$$

For all other integers i we put $b_{i,n} = 0$.

The polynomials are due to Bernstein (1912); Farouki (2012) surveys their history and properties, and Figure 2 shows the cubic ones. The proofs below use two identities of binomial coefficients.

Lemma 2 (Binomial identities). *Let $n \geq 1$ and let i, j be integers, with $\binom{m}{k} = 0$ for $k < 0$ or $k > m$. Then*

$$\binom{n-1}{j} + \binom{n-1}{j-1} = \binom{n}{j}$$

and

$$i\binom{n}{i} = n\binom{n-1}{i-1}.$$

Proposition 3 (Bernstein basis). *For $n \geq 0$, the polynomials $b_{0,n}, \ldots, b_{n,n}$ are a basis of the vector space of real polynomials of degree at most n.*

This is Theorem 1.1 of Floater (2025); Appendix A proves it from scratch.

Proposition 4 (Partition of unity). *For every $t \in [0,1]$, $b_{i,n}(t) \geq 0$ for all i, and*

$$\sum_{i=0}^{n} b_{i,n}(t) = 1.$$

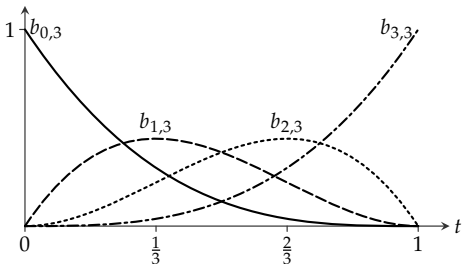


Figure 2
Cubic Bernstein polynomials

Definition 5 (Bézier curve). Let $P_0, \ldots, P_n \in \mathbb{R}^d$. The Bézier curve of degree n with control points $P_0, \ldots, P_n$ is

$$B(t) = \sum_{i=0}^{n} b_{i,n}(t) P_i, \quad t \in [0,1],$$

and its control polygon is the polyline $P_0 P_1 \ldots P_n$.

The definition follows Bézier (1966) in the form of Farin (2002) and Prautzsch et al. (2002).

Proposition 6 (Endpoints). *$b_{i,n}(0) = 1$ if $i = 0$ and 0 otherwise; $b_{i,n}(1) = 1$ if $i = n$ and 0 otherwise. Hence $B(0) = P_0$ and $B(1) = P_n$.*

Corollary 7 (Convex hull). *Every point $B(t)$, $t \in [0,1]$, lies in the convex hull of the control points $P_0, \ldots, P_n$.*

Proposition 8 (Affine invariance). *For every affine map $A(x) = Mx + v$ and every $t \in [0,1]$,*

$$A(B(t)) = \sum_{i=0}^{n} b_{i,n}(t) A(P_i).$$

Transforming the curve is the same as transforming its control points.

Corollary 7 keeps a curve inside the region its control polygon spans, and Proposition 8 is why the exporters and the Matplotlib adapter can move, scale or rotate a curve by moving its control points alone.

`BernsteinBasis`

`BernsteinBasis(`⟨*int*⟩`)(`⟨*float*⟩`)`
`BernsteinBasis(`⟨*int*⟩`).matrix(`⟨*values*⟩`)`

The basis of one degree, in `bezierkit.bezier.basis`. Calling it at t returns the values $b_{i,n}(t)$ for $i = 0, \ldots, n$ as an array; `matrix()` returns one such row per parameter value.

5.2 Curves

`BezierCurve`

`BezierCurve(`⟨*points*⟩`, *, evaluator=None)`
`BezierCurve.linear(p0, p1)`
`BezierCurve.quadratic(p0, p1, p2)`
`BezierCurve.cubic(p0, p1, p2, p3)`

An immutable Bézier curve of any degree $n \geq 0$ and dimension $d \geq 1$, over $[0,1]$. `points` is a sequence of `Point`s, a `PointSet`, a $(n+1) \times d$ array or a `ControlPolygon`; the control points must share one dimension.

`degree` The degree n.

`dimension` The dimension d.

`control_points` The control points as a `PointSet`.

`at()` $B(t)$ as a `Point`; a curve object is also callable, `curve(t)`.

`at_many()` B at many parameters, as a `PointSet`.

`evaluator` The evaluation strategy (see below).

The operations on a curve, `derivative()`, `split()`, `segment()` and `reversed()`, are the subject of Section 6.

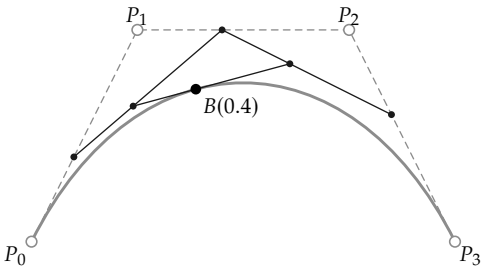


Figure 3
de Casteljau at $t = 0.4$

5.3 Evaluation

de Casteljau's algorithm evaluates $B(t)$ by repeated linear interpolation.

Definition 9 (de Casteljau points). For a parameter t put $P_i^{(0)} = P_i$ and, for $r = 1, \dots, n$,

$$P_i^{(r)} = (1-t)P_i^{(r-1)} + tP_{i+1}^{(r-1)}, \quad i = 0, \dots, n-r.$$

Each round averages neighbouring points, so the polygon shrinks by one point; after n rounds one point is left (Figure 3). The algorithm goes back to Paul de Casteljau's work at Citroën around 1959 (Müller, 2024).

Theorem 10 (de Casteljau). *For $0 \le r \le n$ and $0 \le i \le n-r$,*

$$P_i^{(r)} = \sum_{j=0}^{r} b_{j,r}(t) P_{i+j}.$$

In particular $P_0^{(n)} = B(t)$.

The statement is Theorems 1.6 and 1.7 of Floater (2025).

Every intermediate point is a convex combination of the control points, so the algorithm never forms large cancelling terms; it is the numerically stable choice for high degrees. Its cost is $O(n^2)$ per parameter.

`DeCasteljauEvaluator` The default evaluation strategy, in `bezierkit.bezier.evaluation`: the algorithm above, applied to a whole batch of parameters at once with NumPy.

`BernsteinEvaluator` Multiplies the Bernstein matrix by the control points. It is faster for low degrees and large batches but sums terms that can cancel at high degree. By Theorem 10 both strategies compute the same polynomial.

```
from bezierkit import BezierCurve
from bezierkit.bezier.evaluation import BernsteinEvaluator

fast = BezierCurve(curve.control_points, evaluator=BernsteinEvaluator())
print(fast.at(0.5))   # Point(coords=(2.0, 1.5)), as with the default
```

The repository's `benchmarks/` directory compares the two on batches of 400, 10,000 and 100,000 parameter values.

5.4 Degree elevation

A curve of degree n is also a curve of degree $n+1$; the new control points are convex combinations of neighbouring old ones.

Lemma 11 (Degree elevation identity). *For $n \geq 0$, $0 \leq i \leq n$ and every t,*

$$b_{i,n}(t) = \frac{n+1-i}{n+1} b_{i,n+1}(t) + \frac{i+1}{n+1} b_{i+1,n+1}(t).$$

Proposition 12 (Degree elevation). *Let B have control points $P_0, \ldots, P_n$. For $k = 0, \ldots, n+1$ put*

$$Q_k = \frac{k}{n+1} P_{k-1} + \left(1 - \frac{k}{n+1}\right) P_k,$$

where P_{-1} and P_{n+1} may be chosen arbitrarily, since their coefficients are zero. Then

$$B(t) = \sum_{k=0}^{n+1} b_{k,n+1}(t) Q_k \quad \text{for all } t.$$

The curve and its parameterization are unchanged.

The formula is standard (Farin, 2002; Prautzsch et al., 2002); the proof in Appendix A is self-contained. Proposition 22 applies it to lines and quadratics, the cases the package needs.

6 Derivatives, reversal and subdivision

Each operation acts on the control points alone and returns a new curve that keeps the evaluator of the original.

6.1 Derivatives

The derivative $B'(t)$ of a Bézier curve is the derivative of its coordinate functions, which are polynomials in t; it is a vector in $\mathbb{R}^d$.

Lemma 13 (Derivative of the Bernstein polynomials). *For $n \geq 1$ and every integer i,*

$$b'_{i,n}(t) = n\big[b_{i-1,n-1}(t) - b_{i,n-1}(t)\big].$$

This is Lemma 1.4 of Floater (2025).

Theorem 14 (Hodograph). *The derivative of a degree-n Bézier curve ($n \geq 1$) is the degree $n-1$ Bézier curve*

$$B'(t) = \sum_{i=0}^{n-1} b_{i,n-1}(t) \cdot n(P_{i+1} - P_i).$$

The curve of control points $n(P_{i+1} - P_i)$ is the hodograph of B (Theorem 1.8 of Floater (2025); see also Farin (2002)).

Corollary 15 (End tangents). *$B'(0) = n(P_1 - P_0)$ and $B'(1) = n(P_n - P_{n-1})$.*

This is why a curve leaves P_0 in the direction of P_1 and arrives at P_n from P_{n-1}.

`BezierCurve.derivative` `derivative(order=1)`

The `order`-th derivative as a `BezierCurve`, by applying Theorem 14 `order` times. The derivative of a constant (degree 0) is the zero curve of degree 0; `order=0` returns the curve itself.

```
d = curve.derivative()
print(list(d.control_points))
# [Point(coords=(3.0, 6.0)), Point(coords=(6.0, 0.0)), Point(coords=(3.0,
-6.0))]
print(d.at(0.5))   # Point(coords=(4.5, 0.0)): the tangent at the top is
horizontal
```

6.2 Reversal

Lemma 16 (Symmetry). *$b_{i,n}(1-t) = b_{n-i,n}(t)$ for all i and t.*

Proposition 17 (Reversal). *The curve with control points $P_n, \ldots, P_0$ is $t \mapsto B(1-t)$.*

`BezierCurve.reversed` The curve traced in the opposite direction, by Proposition 17.

6.3 Subdivision

Running de Casteljau's algorithm at $t = c$ does more than evaluate: the first points of each round form the control polygon of the part of the curve before c, and the last points that of the part after it (Figure 4).

Theorem 18 (Subdivision). *Let $c \in [0,1]$, and compute the de Casteljau points of Definition 9 at $t = c$. Put $L_j = P_0^{(j)}$ and $R_j = P_j^{(n-j)}$ for $j = 0, \ldots, n$. Then for every $s \in [0,1]$*

$$B(cs) = \sum_{j=0}^{n} b_{j,n}(s) L_j$$

and

$$B(c + (1-c)s) = \sum_{j=0}^{n} b_{j,n}(s) R_j.$$

This is a standard result; see Floater (2025) (Section 8.4, where it is derived from the blossom) and Farin (2002).

`BezierCurve.split` `split(⟨float⟩)`
`BezierCurve.segment` `segment(t0, t1)`

`split(c)` returns the two curves of Theorem 18, each of the same degree and parameterized over $[0,1]$. `segment(t0, t1)` returns the part of the curve between t_0 and t_1, reparameterized over $[0,1]$; when $t_0 = t_1$ it is the single point $B(t_0)$, a curve of degree 0. Parameters outside $[0,1]$, or $t_0 > t_1$, raise.

`segment()` splits twice: first at t_1, keeping the left part, then that part at t_0/t_1, keeping the right part.

Corollary 19 (Segment extraction). *For $0 \le t_0 < t_1 \le 1$ the curve returned by `segment(t0, t1)` is $s \mapsto B(t_0 + (t_1 - t_0)s)$.*

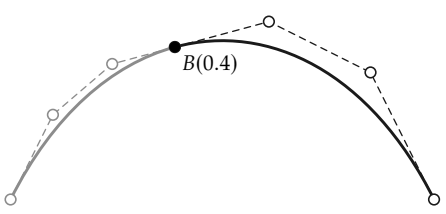


Figure 4
Splitting the cubic at $t = 0.4$

```
left, right = curve.split(0.4)
print(left.at(1.0), right.at(0.0))          # both B(0.4) = (1.552, 1.44)
print(curve.segment(0.25, 0.75).at(0.5))    # B(0.5) = (2.0, 1.5)
```

7 Cubic segments and paths

Renderers draw cubics: TikZ's `.. controls ..`, SVG's `C` and Matplotlib's `CURVE4` all take four control points. `CubicBezierSegment` is that cubic as a value object, and `PiecewiseBezier` joins cubics into paths.

7.1 Cubic segments

`CubicBezierSegment` `CubicBezierSegment(p0, p1, p2, p3, *, fit_error=None)`

An immutable cubic with its four control points as the fields `p0`, `p1`, `p2`, `p3` (also `control_points`, and iteration). It evaluates, differentiates, splits, restricts and reverses like a `BezierCurve`, but `split()`, `segment()` and `reversed()` return cubic segments again. `segment(t, t)` returns the degenerate cubic whose four controls are all $B(t)$.

`fit_error` — *float | None* · default `None`
The measured error of a fitted segment (Section 9); kept through split, segment and reversal, and ignored by equality.

`as_curve()` — Convert to a degree-3 `BezierCurve` without changing a control point.

`from_curve(curve)` — Convert a degree-3 `BezierCurve` without changing a control point.

`from_line()` — The exact cubic of a line (Proposition 22).

`from_quadratic()` — The exact cubic of a quadratic (Proposition 22).

7.2 Tight bounding boxes

Definition 20 (Axis-aligned bounding box). The axis-aligned bounding box of a non-empty bounded set $K \subset \mathbb{R}^d$ is the smallest box

$$[\alpha_1, \beta_1] \times \ldots \times [\alpha_d, \beta_d]$$

that contains K, where $\alpha_k = \inf\{x_k : x \in K\}$ and $\beta_k = \sup\{x_k : x \in K\}$.

The convex hull of the control points contains a curve (Corollary 7) but can be far larger than it. `bounding_box` returns the box of Definition 20 for the curve itself (Figure 5). Its candidates come from the roots of the derivative of each coordinate.

Proposition 21 (Bounding box of a cubic). *Let B be the cubic with control points $P_0, \ldots, P_3 \in \mathbb{R}^d$, and put*

$$a = -P_0 + 3P_1 - 3P_2 + P_3, \quad b = 3P_0 - 6P_1 + 3P_2, \quad c = 3(P_1 - P_0).$$

Then $B(t) = at^3 + bt^2 + ct + P_0$. For each coordinate k let T_k consist of $0, 1$ and the roots in $(0, 1)$ of

$$B_k'(t) = 3a_k t^2 + 2b_k t + c_k$$

when this polynomial is not identically zero; when it is, $T_k = \{0, 1\}$. Then the coordinates of the bounding box of $B([0, 1])$ are

$$\alpha_k = \min_{t \in T_k} B_k(t), \quad \beta_k = \max_{t \in T_k} B_k(t).$$

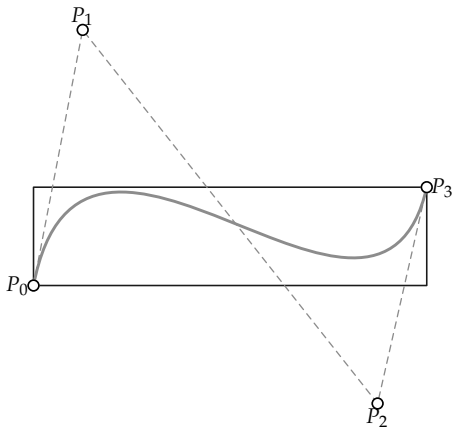


Figure 5
Tight bounding box of a cubic

`CubicBezierSegment.bounding_box` `(low, high)`: two points holding the per-axis minimum and maximum of the curve, found by evaluating it at the candidate parameters of Proposition 21.

7.3 Degree elevation

Degree elevation (Proposition 12) writes a curve of degree n as a curve of degree $n+1$ with the same parameterization. The package needs it to write lines and quadratics as cubics, so that every renderer receives four control points. Applying Proposition 12 once or twice gives the controls below.

Proposition 22 (Lines and quadratics as cubics). *The line from P_0 to P_3, parameterized by $(1-t)P_0 + tP_3$, is the cubic with controls*

$$P_0, \quad P_0 + \frac{1}{3}(P_3 - P_0), \quad P_0 + \frac{2}{3}(P_3 - P_0), \quad P_3.$$

The quadratic with controls P_0, C, P_3 is the cubic with controls

$$P_0, \quad P_0 + \frac{2}{3}(C - P_0), \quad P_3 + \frac{2}{3}(C - P_3), \quad P_3.$$

In both cases the curve and its parameterization are unchanged.

`to_cubic` `to_cubic(⟨curve⟩)`

In `bezierkit.bezier`. Turns a linear, quadratic or cubic `BezierCurve` into a `CubicBezierSegment` by Proposition 22 (a segment is returned as is); higher degrees raise `DegreeError`.

`line_to_cubic` `line_to_cubic(p0, p3)`

The cubic of the line from `p0` to `p3`.

`quadratic_to_cubic` `quadratic_to_cubic(p0, control, p3)`

The cubic of the quadratic with the given controls.

```python
from bezierkit.bezier import to_cubic

q = BezierCurve.quadratic(Point(0, 0), Point(3, 6), Point(9, 0))
print(to_cubic(q).control_points)
# (Point(coords=(0.0, 0.0)), Point(coords=(2.0, 4.0)),
#  Point(coords=(5.0, 4.0)), Point(coords=(9.0, 0.0)))
```

7.4 Piecewise paths

`PiecewiseBezier` `PiecewiseBezier(⟨segments⟩, *, closed=False, continuity_tolerance=1e-9)`
`PiecewiseBezier.compound(⟨paths⟩)`

A path of cubic segments. Consecutive segments must meet: each segment's `p3` must lie within `continuity_tolerance` of the next one's `p0`, and a closed path's last `p3` within it of the first `p0`; otherwise `ValueError` is raised. `closed` is metadata for exporters (SVG `Z`, TikZ `cycle`): it never adds a closing segment.

The package parameterizes a path uniformly. This is a convention of the package, not a standard notion: a path of m segments $S_0, \ldots, S_{m-1}$ is the map

$$P(t) = S_k(mt - k), \quad k = \min(\lfloor mt \rfloor, m - 1), \quad t \in [0, 1].$$

Each segment takes an equal share $1/m$ of $[0, 1]$, regardless of its length.

`segments` The segments in order.
`control_points` Their control points.
`subpaths` The `BezierSubpaths`.
`at(t)` Evaluate under the uniform parameterization.
`at_many(values)` Evaluate at many parameters.
`split(t)` Divide the path at `t`; a parameter inside a segment splits that segment (Theorem 18).
`segment(t0, t1)` Keep the part between two parameters.
`reversed()` Reverse the order of the segments and each segment.
`closed` Closure flag.
`is_compound` Whether there are several subpaths.
`dimension` The dimension d.

```python
from bezierkit import CubicBezierSegment, PiecewiseBezier

path = PiecewiseBezier([
    CubicBezierSegment.from_line(Point(0, 0), Point(2, 0)),
    CubicBezierSegment.from_line(Point(2, 0), Point(2, 4)),
])
print(path.at(0.75))                      # Point(coords=(2.0, 2.0))
print(path.segment(0.25, 0.75).at(1.0))   # Point(coords=(2.0, 2.0))
```

`compound()` gathers several paths into one, keeping their subpaths separate, as for a shape with a hole. A compound path has no single parameterization: evaluating, splitting or restricting it raises `ValueError`; reversal and export work. Splitting or restricting a closed path also raises, except for the whole path or a single point, since a closed path has no first and last point to keep.

7.5 Continuity

Two curves meet at a join. Let $h_S, h_T > 0$, let $S : [u_0 - h_S, u_0] \to \mathbb{R}^d$ and $T : [u_0, u_0 + h_T] \to \mathbb{R}^d$ be continuously differentiable as often as needed (with one-sided derivatives at the ends of their intervals), and let C be the curve that equals S on the first interval and T on the second.

Definition 23 (Parametric continuity). For $k \geq 0$, the curve C is C^k at u_0 if $S^{(j)}(u_0) = T^{(j)}(u_0)$ for $j = 0, \ldots, k$.

Definition 24 (Geometric continuity G^1). The curve C is G^1 at u_0 if $S(u_0) = T(u_0)$, the tangent vectors $S'(u_0)$ and $T'(u_0)$ are non-zero, and $T'(u_0) = \lambda S'(u_0)$ for some $\lambda > 0$; that is, the two unit tangents agree.

Parametric and geometric continuity are the standard notions for joined curves (Farin, 2002; Prautzsch et al., 2002); the geometric notion of every order k is treated by Barsky & DeRose (1989). This manual uses C^0, C^1 and G^1 only. A C^1 join with non-zero tangents is G^1.

Proposition 25 (Continuity of cubic joins). *Let S and T be the cubics with controls $P_0, \dots, P_3$ and $Q_0, \dots, Q_3$, evaluated at $(u - u_0 + h_S)/h_S$ and $(u - u_0)/h_T$ respectively.*

1. *C is C^0 at u_0 if and only if $P_3 = Q_0$.*
2. *If $P_3 = Q_0$, then C is C^1 at u_0 if and only if*

$$\frac{P_3 - P_2}{h_S} = \frac{Q_1 - Q_0}{h_T}.$$

3. *If $P_3 = Q_0$, $P_3 \neq P_2$ and $Q_1 \neq Q_0$, then C is G^1 at u_0 if and only if $Q_1 - Q_0 = \mu(P_3 - P_2)$ for some $\mu > 0$.*
4. *Under the uniform parameterization of `PiecewiseBezier`, $h_S = h_T$, so the condition in 2 reads $P_3 - P_2 = Q_1 - Q_0$.*

`PiecewiseBezier` enforces only C^0, a shared endpoint. Whether a join should also be smooth is the caller's decision: a kink in an indifference curve or a corner in a polyline is meant to stay.

8 Constructions and Hermite interpolation

A cubic is fixed by its endpoints and the derivatives there. The constructions in this chapter turn such endpoint conditions into control points, which is how economic curves known by a few values and slopes become Bézier curves.

8.1 Endpoint constructions

Each construction is a frozen dataclass with a `build()` method, in `bezierkit.construction`.

`EndpointDerivatives` `EndpointDerivatives(start, end, start_derivative, end_derivative).build()`

The cubic with $B(0) =$ `start`, $B(1) =$ `end`, $B'(0) =$ `start_derivative` and $B'(1) =$ `end_derivative`, in any dimension.

Proposition 26 (Endpoint conditions). *Let $P_0, P_3, D_0, D_1 \in \mathbb{R}^d$. The cubic Bézier curve with controls*

$$P_0, \quad P_0 + D_0/3, \quad P_3 - D_1/3, \quad P_3$$

satisfies $B(0) = P_0$, $B(1) = P_3$, $B'(0) = D_0$ and $B'(1) = D_1$, and it is the only polynomial curve of degree at most 3 that does.

`TangentDirections` `TangentDirections(start, end, start_direction, end_direction, start_handle=1.0, end_handle=1.0).build()`

Endpoint tangents given as directions, with the handle lengths $|P_1 - P_0| =$ `start_handle` and $|P_3 - P_2| =$ `end_handle`. The directions are normalized, so only their orientation matters; handles must be finite and non-negative. By Proposition 26, $B'(0) = 3 \cdot$ `start_handle` times the unit start direction.

`PlanarSlopes` `PlanarSlopes(start, end, start_slope, end_slope, start_handle=1.0, end_handle=1.0).build()`

A planar cubic whose ends have the slopes $\mathrm{d}y/\mathrm{d}x$ given, built as `TangentDirections` with directions $(1, m)$. Slopes must be finite; a vertical tangent needs `TangentDirections`.

```
from bezierkit.construction import PlanarSlopes

demand = PlanarSlopes(
    start=Point(0, 5), end=Point(5, 0), start_slope=-2, end_slope=-0.3,
).build()
print(demand.control_points[1])   # (0.447, 4.106): one unit from (0, 5)
along slope -2
```

8.2 Hermite interpolation

Cubic Hermite interpolation matches a function and its derivative at both ends of an interval.

Definition 27 (Cubic Hermite interpolant). Let f be differentiable on $[x_0, x_1]$ with $x_0 \neq x_1$. A cubic Hermite interpolant of f is a polynomial H of degree at most 3 with

$$H(x_i) = f(x_i), \quad H'(x_i) = f'(x_i), \quad i = 0, 1.$$

For a curve, interpolate each coordinate.

This is the case of two nodes of the Hermite polynomial in Section 3.4 of Burden & Faires (2011), where it is Definition 3.8. Such a polynomial exists and is unique:

Proposition 28 (Existence and uniqueness). *The cubic Hermite interpolant of Definition 27 exists and is unique. With $h = x_1 - x_0$, $f_i = f(x_i)$ and $m_i = f'(x_i)$ it is*

$$H(x) = \sum_{i=0}^{3} b_{i,3}((x - x_0)/h)c_i,$$

where the coefficients are

$$c_0 = f_0, \quad c_1 = f_0 + hm_0/3, \quad c_2 = f_1 - hm_1/3, \quad c_3 = f_1.$$

The same statement, for any number of nodes, is Theorem (2.1.5.2) of Stoer & Bulirsch (2002). Over a parameter interval $[t_0, t_1]$ of width $h = t_1 - t_0$, the Bézier segment is evaluated at $s = (t - t_0)/h$, which rescales derivatives by h.

`parametric_hermite` `parametric_hermite(p0, p3, derivative0, derivative1, *, t0=0.0, t1=1.0)`

The cubic segment with controls P_0, $P_0 + hD_0/3$, $P_3 - hD_1/3$, P_3, in `bezierkit.interpolation`. $h = 0$ raises `ValueError`; a negative h reverses the interval.

Proposition 29 (Hermite interpolation). *Let $H(t) = B((t - t_0)/h)$ for the segment returned by `parametric_hermite`. Then $H(t_0) = P_0$, $H(t_1) = P_3$, $H'(t_0) = D_0$ and $H'(t_1) = D_1$.*

`graph_hermite` `graph_hermite(*, x0, x1, y0, y1, m0, m1)`

The interpolant of a graph $y = f(x)$ from the values $y_0 = f(x_0), y_1 = f(x_1)$ and slopes $m_0 = f'(x_0)$, $m_1 = f'(x_1)$: the parametric form with $P = (x, f(x))$, $D = (1, f'(x))$ and $t = x$. Its controls are

$$(x_0, y_0), \quad \left(x_0 + \frac{h}{3}, y_0 + hm_0/3\right), \quad \left(x_1 - \frac{h}{3}, y_1 - hm_1/3\right), \quad (x_1, y_1),$$

with $h = x_1 - x_0$ (Figure 6).

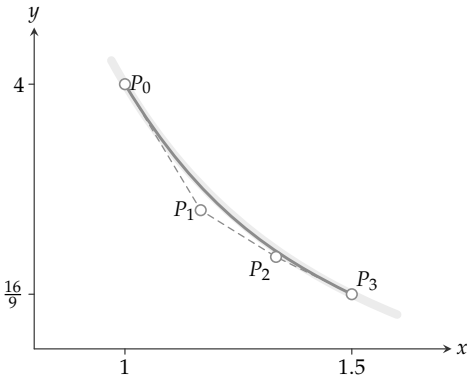


Figure 6
Hermite interpolant of $4/x^2$

Proposition 30 (Graph form). *The segment of* `graph_hermite` *is the graph of the cubic Hermite interpolant H of f on $[x_0, x_1]$: it is $s \mapsto (x_0 + hs, H(x_0 + hs))$, $s \in [0, 1]$.*

```
from bezierkit.interpolation import graph_hermite

# f(x) = 4 / x^2 on [1, 1.5]
segment = graph_hermite(x0=1, x1=1.5, y0=4, y1=16 / 9, m0=-8, m1=-64 / 27)
```

8.3 Interpolation error

Theorem 31 (Hermite error bound). *Let f be four times continuously differentiable on $[x_0, x_1]$ with $x_0 < x_1$, let H be its cubic Hermite interpolant (Definition 27), and put $h = x_1 - x_0$ and $M = \max|f^{(4)}|$ on the interval. For every $x \in [x_0, x_1]$ there is a $\xi \in [x_0, x_1]$ with*

$$f(x) - H(x) = \frac{f^{(4)}(\xi)}{24}(x - x_0)^2(x - x_1)^2,$$

and therefore

$$\max_{x \in [x_0, x_1]} |f(x) - H(x)| \le \frac{Mh^4}{384}.$$

The error formula is Theorem 3.9 of Burden & Faires (2011), whose proof is left to Exercise 3.4.11 there, and Theorem (2.1.5.9) of Stoer & Bulirsch (2002). The bound is sharp: for $f(x) = x^4$ on $[0, 1]$ the error is $x^2(x-1)^2$, whose maximum $1/16$ at $x = 1/2$ equals $Mh^4/384$ with $M = 24$.

Halving the interval divides the bound by 16, which is what adaptive fitting exploits (Section 9). For $f(x) = 4/x^2$ on $[1, 1.5]$, $f^{(4)}(x) = 480/x^6$, so $M = 480$ and the bound is $480 \cdot 0.5^4/384 \approx 0.078$.

`hermite_error_bound` `hermite_error_bound(max_fourth_derivative, t0, t1)`

The bound $M\,|t_1 - t_0|^4\,/384$ of Theorem 31, in `bezierkit.fitting`. M must be finite and non-negative.

The bound is for a scalar function. For a curve $t \mapsto (x(t), y(t))$, apply it to each coordinate. In $\mathbb{R}^d$ the Euclidean error is at most $\sqrt{d}$ times the largest coordinate bound, because $\|v\|_2 \le \sqrt{d}\|v\|_\infty$ for $v \in \mathbb{R}^d$ (Section 2.2 of Golub & Van Loan (2013)).

9 Fitting

Fitting turns a smooth function or a list of sampled points into a `PiecewiseBezier` whose error is measured and reported, segment by segment, in the coordinate units of the curve. The functions live in `bezierkit.fitting`.

9.1 Adaptive Hermite fitting

The fit of a parametric curve $C : [t_0, t_1] \to \mathbb{R}^d$ on an interval $[a, b]$ is the cubic Hermite interpolant of Definition 27, taken for each coordinate (Proposition 29). The package measures its error at q equally spaced probe parameters $\tau_j = a + j(b-a)/(q-1)$, $j = 0, \ldots, q-1$, where q is `error_samples`:

$$e = \max_{0 \le j \le q-1} \left\| C(\tau_j) - H(\tau_j) \right\|_2 .$$

This measured error is a convention of the package; it is never larger than the true maximum error of the segment.

`fit_parametric` `fit_parametric(`⟨*function*⟩`, `⟨*derivative*⟩`, *, t0, t1, tolerance, max_depth=20, max_segments=4096, error_samples=9)`

Fit a parametric curve $C(t)$, given as a function returning `Points` and its derivative returning `Vectors`, over $[t_0, t_1]$. The algorithm is a recursive bisection:

1. Build the Hermite segment on the current interval from C and C' at its ends (Proposition 29).
2. Measure its error e.
3. If e is at most `tolerance`, keep the segment, recording e as its `fit_error`. Otherwise halve the interval and fit each half.

Every accepted segment therefore meets the tolerance at its probe parameters. If an interval still fails at depth `max_depth`, or keeping it would exceed `max_segments`, `ToleranceNotMet` is raised rather than a worse path returned.

`tolerance` *float* — Allowed error, in coordinate units; must be positive.

`max_depth` *int* · default 20 — Largest number of halvings of the starting interval.

`max_segments` *int* · default 4096 — Largest number of segments in the result.

`error_samples` *int* · default 9 — Probe parameters per segment, at least 3.

`fit_graph` `fit_graph(`⟨*function*⟩`, `⟨*derivative*⟩`, *, x0, x1, tolerance, ...)`

Fit the graph $y = f(x)$ for $x \in [x_0, x_1]$: `fit_parametric` with $C(x) = (x, f(x))$ and $C'(x) = (1, f'(x))$, the same options, and the error measured as a Euclidean distance in the (x, y) plane.

```
from bezierkit.fitting import fit_graph

path = fit_graph(lambda x: 4 / x**2, lambda x: -8 / x**3,
                 x0=0.8, x1=3, tolerance=1e-3)
print(len(path.segments))                  # 10
print(max(s.fit_error for s in path))      # 0.000676
```

Segments are short where the curve bends sharply and long where it is nearly straight (Figure 7). The Hermite error bound predicts how deep the bisection has to go:

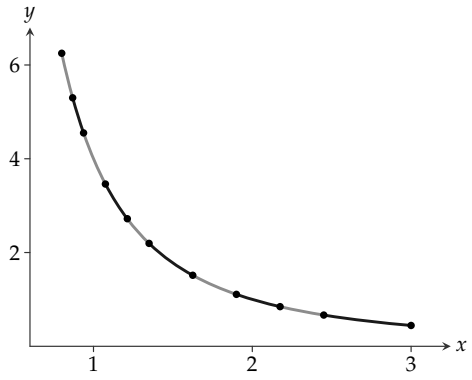


Figure 7
Adaptive fit of $4/x^2$

Corollary 32 (Depth of adaptive fitting). *Let each coordinate of C be four times continuously differentiable on $[t_0, t_1]$ with $|C_k^{(4)}| \le M$, put $L = |t_1 - t_0|$, and let d be the dimension and $\varepsilon > 0$ the tolerance. Put*

$$k^* = \max\left(0, \left\lceil \frac{1}{4} \log_2\left(\sqrt{d} M \frac{L^4}{384\varepsilon} \right) \right\rceil\right),$$

with $k^* = 0$ when $M = 0$. Every interval at a depth $k \ge k^*$ passes the error test. Hence if `max_depth` $\ge k^*$ and `max_segments` $\ge 2^{k^*}$, `fit_parametric` does not raise `ToleranceNotMet`, and no interval deeper than k^* is created.

The measured error is a lower bound for the true maximum error of a segment: it is checked only at the probe parameters. Increase `error_samples` when the curve could wiggle between them.

9.2 Polyline simplification

Definition 33 (Distance to a segment). For $x, a, b \in \mathbb{R}^d$, the distance from x to the segment $[a, b]$ is

$$\operatorname{dist}(x, [a, b]) = \min_{\lambda \in [0,1]} \|x - a - \lambda(b - a)\|_2.$$

Polyline simplification by recursive subdivision is the Ramer–Douglas–Peucker algorithm (Douglas & Peucker, 1973; Ramer, 1972). `fit_polyline` uses the distance of Definition 33 throughout.

`fit_polyline` `fit_polyline(`⟨*points*⟩`, *, tolerance, closed=False, duplicate_tolerance=1e-12, preserve_corners=True, corner_angle=pi/4)`

Simplify sampled points into a path of straight chords, each written as an exact cubic (Proposition 22). The steps are:

1. Drop each point within `duplicate_tolerance` of the one kept before it. A closed polyline also drops a final copy of its first point, and is rotated so that it starts at its lexicographically smallest point, which makes the output independent of where the samples began.
2. Keep the end points and, with `preserve_corners`, every vertex where the direction turns by at least `corner_angle` radians.
3. Between consecutive kept vertices, apply the recursive step: if the vertex farthest from the chord is within `tolerance` of it, the chord replaces the whole run; otherwise keep that vertex and recurse on both sides.

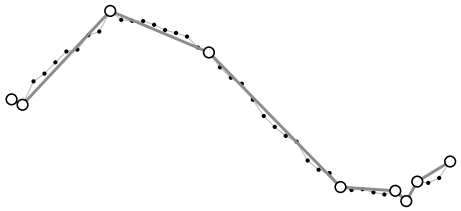


Figure 8
Noisy samples simplified

Each chord's `fit_error` is the largest distance from the vertices it replaced to the chord (Figure 8).

The recursion makes at most as many levels as there are vertices and scans each vertex once per level, so it is quadratic in the number of vertices in the worst case. Hershberger & Snoeyink (1994) give an $O(n \log n)$ implementation of the algorithm.

Proposition 34 (Polyline tolerance). *Let $v_0, \ldots, v_N$ be the vertices left by step 1 (for a closed polyline $v_N = v_0$), let $0 = a_0 < \ldots < a_m = N$ be the indices of the vertices that the output keeps, and let ε be `tolerance`. For every j and every i with $a_j \le i \le a_{j+1}$,*

$$\operatorname{dist}\Big(v_i, \Big[v_{a_j}, v_{a_{j+1}}\Big]\Big) \le \varepsilon.$$

In particular the `fit_error` of every output segment is at most ε.

Corollary 35 (Deviation from the polyline). *With the notation of Proposition 34:*

1. *Every point of the polyline $v_0 v_1 \ldots v_N$ lies within ε of one chord $\Big[v_{a_j}, v_{a_{j+1}}\Big]$ of the output.*
2. *Every input point lies within $\varepsilon + \delta$ of a chord, where δ is `duplicate_tolerance`; the points left by step 1 lie within ε.*

`maximum_polyline_deviation`

`maximum_polyline_deviation(`⟨*points*⟩`, `⟨*path*⟩`)`

The largest distance from any of the points to the nearest chord P_0P_3 of the path's segments: a check of Proposition 34 and Corollary 35 that also covers the points step 1 dropped.

Like any method that sees only samples, `fit_polyline` bounds the deviation from the samples, not from an unknown continuous curve between them; sample more densely for a closer match.

10 Level sets

Indifference curves, isoquants and contour lines are level sets of a function of two variables.

10.1 Level sets and their tangents

Definition 36 (Level set). For $F : \mathbb{R}^2 \to \mathbb{R}$ and a level $c \in \mathbb{R}$, the level set is

$$L_c = \{(x, y) : F(x, y) = c\}.$$

For a continuously differentiable F, write F_x and F_y for its partial derivatives and $\nabla F = \big(F_x, F_y\big)$ for its gradient. Near a point with $\nabla F \neq 0$ the level set is a curve. This follows from the implicit function theorem, which is quoted here in the form that is needed.

Theorem 37 (Implicit function theorem). *Let $\Omega \subset \mathbb{R}^2$ be open, $F : \Omega \to \mathbb{R}$ continuously differentiable, $p \in \Omega$, $F(p) = c$ and $F_y(p) \neq 0$. Then there are open intervals I and J with $p_x \in I$, $p_y \in J$ and $I \times J \subset \Omega$, and a continuously differentiable $\varphi : I \to J$ with $\varphi(p_x) = p_y$, such that for $(x, y) \in I \times J$*

$$F(x, y) = c \quad \text{if and only if} \quad y = \varphi(x).$$

The theorem holds for any number of variables; see Theorem 9.28 of Rudin (1976), Theorem 13.7 of Apostol (1974) (second edition) and Theorem 9.2 of Munkres (1991). It is not proved here.

Theorem 38 (Tangent of a level set). *Let F be continuously differentiable on an open set $\Omega \subset \mathbb{R}^2$, $p \in \Omega$, $F(p) = c$ and $\nabla F(p) \neq 0$. Then there are an open neighbourhood N of p and a continuously differentiable curve $\gamma : I \to \mathbb{R}^2$ on an open interval I with $\gamma(I) = L_c \cap N$, $\gamma(u_0) = p$ for some $u_0 \in I$, and $\gamma'(u) \neq 0$ for all $u \in I$, such that $\gamma'(u)$ is parallel to $\left(F_y, -F_x\right)$ at $\gamma(u)$. The vector $\left(F_y, -F_x\right)$ is orthogonal to ∇F.*

The tangent $\left(F_y, -F_x\right)$ needs no division, so a vertical tangent ($F_y = 0$) needs no special case, although the slope $\mathrm{d}y/\,\mathrm{d}x = -F_x/F_y$ of the graph form is infinite there.

10.2 Tracing

`trace_implicit` turns level sets into cubic paths without solving for y, so it handles curves that turn back, close up or break into several pieces.

`trace_implicit` `trace_implicit(⟨function⟩, *, levels, viewport, resolution=(101, 101), tolerance=1e-2, gradient=None)`

Trace $F(x, y) = c$ for each level c inside the rectangle `viewport` $= (x_{\min}, x_{\max}, y_{\min}, y_{\max})$, in `bezierkit.implicit`.

`function` F, called with two floats; it must be finite at every grid point, or `ValueError` is raised.

`levels` One level or an iterable of finite levels.

`resolution` — *(int, int)* · default `(101, 101)` — Grid points along x and y, at least 2 each.

`tolerance` — *float* · default `1e-2` — Allowed deviation, in coordinate units, of the simplified and curved segments.

`gradient` — *callable | None* · default `None` — $\nabla F = \left(F_x, F_y\right)$, as a pair or a `Vector`. When given, straight pieces are bent into cubics that follow the curve's tangent.

`trace_sampled` `trace_sampled(⟨xs⟩, ⟨ys⟩, ⟨values⟩, *, levels, tolerance=1e-2, gradient=None)`

New: v1.1.0 Trace $F(x, y) = c$ on a grid whose values are already computed, in `bezierkit.implicit`. `values[row, column]` is F at `(xs[column], ys[row])`, so a vectorized F can fill the whole grid in one call instead of one call per point. `levels`, `tolerance` and `gradient` are as for `trace_implicit`; `xs` and `ys` must be increasing, and `ValueError` is raised for a mismatched shape or a non-finite value.

The algorithm is marching squares, the two-dimensional analogue of the marching cubes of Lorensen & Cline (1987). A grid cell has corners $v_{00}, v_{10}, v_{11}, v_{01}$, counter-clockwise from the lower left, with values $f_{00}, f_{10}, f_{11}, f_{01}$ of F. A corner is high if its value is at least the level c.

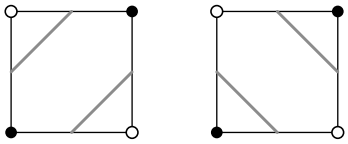


Figure 9
Saddle cell with two centres

Definition 39 (Edge crossing). Let a grid edge run from p to q with $F(p) \geq c > F(q)$ or $F(q) \geq c > F(p)$. Its crossing point is $p + \lambda(q - p)$ with

$$\lambda = \frac{c - F(p)}{F(q) - F(p)},$$

the zero of the linear interpolant of $F - c$ along the edge.

An edge has a crossing exactly when its two corners differ in being high.

1. Sample. Evaluate F on a regular grid of the viewport.
2. March. In each grid cell, find the crossings of its four edges (Definition 39). A cell with two crossings contributes one line piece. A cell with four crossings is a saddle: two opposite corners are high, the other two low, and the rule below decides which crossings to join (Figure 9).
3. Stitch. Join pieces that share an end point into chains. A chain that returns to its start is closed; one that reaches the viewport's edge or a branch point is open. Disconnected components stay separate paths.
4. Simplify. Reduce each chain with `fit_polyline` at `tolerance` (Section 9), without corner preservation.
5. Bend (with `gradient`). Replace each straight piece by a cubic whose end tangents follow the level set (Theorem 38), with handles one third of the chord long. The cubic is kept only if it stays within `tolerance` of the chord; where $\nabla F = 0$ at either end, the piece stays straight.

The saddle rule uses the bilinear interpolant of the four corner values,

$$u(s,t) = (1-s)(1-t)f_{00} + s(1-t)f_{10} + stf_{11} + (1-s)tf_{01}, \quad (s,t) \in [0,1]^2.$$

Proposition 40 (Centre of the bilinear interpolant). *The interpolant u takes the corner values at the corners, and its value at the centre of the cell is the mean of the four corner values,*

$$u\left(\frac{1}{2}, \frac{1}{2}\right) = \frac{f_{00} + f_{10} + f_{11} + f_{01}}{4}.$$

The package calls the centre high when this mean is at least c. In a saddle cell the two corners that are on the same side of c as the centre are then connected through the cell, and each of the other two corners is cut off by a line piece of its own (Figure 9). This is the midpoint decider (Athawale & Johnson, 2019). The asymptotic decider of Nielson & Hamann (1991) compares the value of u at its saddle point instead,

$$u^* = \frac{f_{00}f_{11} - f_{10}f_{01}}{f_{00} - f_{10} - f_{01} + f_{11}},$$

and the two rules differ only when u^* and the mean lie on opposite sides of c.

```
from bezierkit.implicit import trace_implicit

contours = trace_implicit(
    lambda x, y: x**2 * y,
    levels=[1, 2, 4],
    viewport=(0.5, 4, 0, 6),
    resolution=(121, 121),
    tolerance=0.01,
```

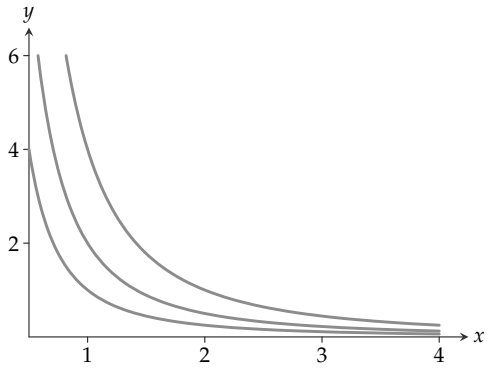


Figure 10
$x^2y = c$ for three values of c

```
    gradient=lambda x, y: (2 * x * y, x**2),
)
print(contours.level_values)                    # (1.0, 2.0, 4.0)
print(len(contours.for_level(4)[0].segments))   # 15
```

10.3 Results

`ContourSet` `trace_implicit` returns a `ContourSet` whose `contours` hold one `LevelContours` per level, in the order given. `level_values` lists the levels, `paths` all paths together, and `for_level(c)` the paths of one level (`KeyError` if it was not traced).

`LevelContours` The `level` and its `paths`, one `PiecewiseBezier` per connected component.

10.4 Limitations

The traced curve is exact only at the grid crossings; between them it is the piecewise-linear marching-squares curve, simplified and bent within `tolerance`. Raise `resolution` and lower `tolerance` for a closer match. Features smaller than a grid cell can be missed, and the saddle rule picks one of the two possible connections. Kinks such as the corner of a Leontief indifference curve are rounded to within the grid spacing, since the grid sees no exact corner.

11 Sampling and export

The exporters write geometry only. They choose no colour, line width, theme, axis, label or canvas, and they never flatten a cubic into line pieces: TikZ gets native `.. controls ..` curves and SVG native `C` commands.

11.1 Sampling

`UniformSampler` `UniformSampler(⟨int⟩).sample(⟨curve⟩)`

Evaluate any curve or path at `count` equally spaced parameters over its domain, both ends included (`count` ≥ 2), in `bezierkit.sampling`.

`Sample` The result of sampling: the read-only parameters `t` and the `points`, with the coordinate arrays `x` and `y` and iteration over `(t, point)` pairs.

```
from bezierkit.sampling import UniformSampler

sample = UniformSampler(5).sample(curve)
print(sample.x)   # [0.      0.90625 2.      3.09375 4.     ]
```

11.2 Rounding and transforms

The text exporters write each coordinate with a fixed number of decimal places `precision`, and accept a `transform` that maps each control point (a `Point`) to a two-dimensional `Point` before it is written, for example from data to page coordinates.

Proposition 41 (Rounding error). *Let B and $\tilde{B}$ be Bézier curves of the same degree n in $\mathbb{R}^d$ with control points P_i and $\tilde{P}_i$, and let $|\tilde{P}_{i,k} - P_{i,k}| \le \varepsilon$ for every i and every coordinate k. Then for every $t \in [0,1]$ and every k*

$$|\tilde{B}_k(t) - B_k(t)| \le \varepsilon, \quad \left\|\tilde{B}(t) - B(t)\right\|_2 \le \sqrt{d}\varepsilon.$$

Rounding every coordinate to p decimal places gives $\varepsilon = \frac{1}{2} \cdot 10^{-p}$.

So with the default `precision=6`, an exported planar curve stays within 0.71×10^{-6} of the original everywhere, not only at its control points. An affine `transform` is exact by Proposition 8; a non-affine one moves the control points correctly but not, in general, the points between them.

11.3 JSON

`dumps` `dumps(`⟨*path*⟩`, *, metadata=None, indent=None)`

In `bezierkit.export.json`. Writes a path in the versioned schema of Table 4, with sorted keys and no NaN or infinity.

`loads` `loads(`⟨*str*⟩`)`

Validates a document and returns a `PathDocument` holding the `path` and its `metadata`. An unknown schema or version, a segment without exactly four points, a point of the wrong dimension, or a non-finite number raises `ValueError`.

`PathDocument` A path together with its caller metadata, as returned by `loads()`.

Table 4
JSON path schema, version 1

Key	Value
`schema`	The string `"bezierkit.path"`
`version`	The integer `1`; readers reject other versions
`dimension`	The dimension d of every point
`subpaths`	A non-empty array of `{"closed": bool, "segments": [...]}`, each segment four points of d numbers
`metadata`	A JSON object, passed through uninterpreted

```
from bezierkit.export.json import dumps, loads

text = dumps(path, metadata={"name": "arch"})
# {"dimension":2,"metadata":{"name":"arch"},"schema":"bezierkit.path",
#  "subpaths":[{"closed":false,"segments":[[[0.0,0.0],
[1.0,2.0],...]]}],"version":1}
assert loads(text).path.segments == path.segments
```

Version 1 is frozen: a new required key, a change in closure semantics or in the control-point layout requires version 2.

11.4 SVG

to_svg_path_data
from_svg_path_data

`to_svg_path_data(⟨path⟩, *, precision=6, transform=None)`
`from_svg_path_data(⟨str⟩)`

In `bezierkit.export.svg`. The path data string (the `d` attribute) of a planar path, using only absolute `M`, `C` and `Z` commands, one `M` per subpath; no SVG document or styling. `from_svg_path_data()` parses that same subset back; relative commands, lines and arcs raise `ValueError`.

```
from bezierkit.export.svg import to_svg_path_data

print(to_svg_path_data(path, precision=2))   # M 0.00 0.00 C 1.00 2.00
3.00 2.00 4.00 0.00
```

11.5 TikZ

to_tikz
segment_to_tikz

`to_tikz(⟨path⟩, *, precision=6, transform=None, options=None)`
`segment_to_tikz(⟨segment⟩, *, precision=6, transform=None, include_start=True)`

In `bezierkit.export.tikz`. One `\draw` command per subpath, each segment written as `.. controls (P1) and (P2) .. (P3)`, closed subpaths ending in `-- cycle`. `options` is inserted verbatim as `\draw[options]`; nothing is chosen by default. `segment_to_tikz()` writes one segment, without the `\draw`, for callers assembling their own commands.

```
from bezierkit.export.tikz import to_tikz

print(to_tikz(path, precision=2, options="thick"))
# \draw[thick] (0.00,0.00) .. controls (1.00,2.00) and (3.00,2.00) ..
(4.00,0.00);
```

Every figure in this manual was made this way: a script builds the curves with bezierkit, writes them with `to_tikz()` into a `standalone` LaTeX document beside the axes and labels, and compiles it to PDF. The `.tex` source of each figure ships with the manual.

11.6 Matplotlib

from_path
to_path
approximate_path

`from_path(⟨path⟩, *, transform=None)`
`to_path(⟨path⟩)`
`approximate_path(⟨path⟩, ⟨transform⟩, *, tolerance, max_depth=20)`

In `bezierkit.adapters.matplotlib`, with the `matplotlib` extra. `from_path()` converts a Matplotlib `Path` exactly: `MOVETO`, `LINETO`, `CURVE3`, `CURVE4` and `CLOSEPOLY` become cubics, lines and quadratics by Proposition 22. An affine `transform` is applied to the control points, which is exact (Proposition 8); a non-affine one raises `ValueError`. `to_path()` converts back, one `CURVE4` per segment.

A non-affine transform, such as a logarithmic axis, does not map cubics to cubics. `approximate_path()` is the explicit opt-in: it subdivides each transformed segment until its midpoints lie within `tolerance / 2` of the chords, then simplifies the points with `fit_polyline` at `tolerance / 2`. It does not claim to be exact.

12 Command-line interface

The `bezierkit` command, installed with the `cli` extra, evaluates, samples and constructs curves from the shell. Its output is a table by default, or JSON or CSV for other programs.

```
bezierkit evaluate --points "0,0" --points "1,2" --points "3,2" --points
"4,0" \
```

```
                    --t 0.5 --order 1 --json
# [{"t":0.5,"value":[4.5,0.0]}]
```

12.1 Control points

Commands that take a curve accept its control points as repeated `--points` options, each a comma-separated list of coordinates (`"1,2"`, or `"1,2,3"` in three dimensions). Without `--points`, they read a JSON object with a `control_points` array from standard input, which is what `construct` writes, so commands can be piped together.

12.2 Commands

`evaluate` `bezierkit evaluate --t ⟨values⟩ [--points ...] [--order ⟨int⟩] [--json]`

Evaluate the curve, or its derivative of order `--order` (default 0), at each `--t`. A value is a number or an inclusive range `start:stop:step`, such as `0:1:0.25`; `--t` may be repeated.

`sample` `bezierkit sample [--points ...] [--count ⟨int⟩] [--format table|csv|json] [--output ⟨path⟩]`

Sample the curve at `--count` (default 50) equally spaced parameters, ends included, as rows of `t`, `x`, `y` (and `z`, or `c3`, `c4`, ... in higher dimensions). `--output` writes to a file instead of standard output.

`construct slopes` `bezierkit construct slopes --start ⟨point⟩ --end ⟨point⟩ --start-slope ⟨float⟩ --end-slope ⟨float⟩ [--start-handle --end-handle]`

`construct tangents` `bezierkit construct tangents --start ... --end ... --start-direction ⟨vector⟩ --end-direction ⟨vector⟩ [...]`

Build a cubic with `PlanarSlopes` or `TangentDirections` (Section 8) and print its control points as JSON.

```
bezierkit construct slopes --start "0,5" --end "5,0" \
                           --start-slope -2 --end-slope -0.3 \
| bezierkit sample --count 3 --format csv
# t,x,y
# 0.0,0.0,5.0
# 0.5,2.3085202413545525,2.272345260462411
# 1.0,5.0,0.0
```

12.3 Errors

Invalid input, such as a malformed point or a parameter outside $[0,1]$, is reported on standard error as `Error: ...` with exit status 1 and no traceback. Put the global option `--debug` before the command to see the Python traceback instead.

Appendix A Proofs

This appendix proves the lemmas, propositions, theorems and corollaries stated in the chapters, in the order they appear. Throughout, $b_{i,n}$ is the Bernstein polynomial of Definition 1 and B the Bézier curve of Definition 5, of degree n with control points $P_0, \ldots, P_n$. Where a result is also proved in the literature, the chapter names the source; the proofs here are written out so that the manual does not depend on a particular edition of a book.

A.1 The Bernstein basis and evaluation

Proof of Lemma 2. First identity. We distinguish the position of j.

- If $j < 0$ or $j > n$, all three coefficients vanish, since $j > n$ implies $j - 1 > n - 1$.
- If $j = 0$, the left side is $1 + 0 = 1 = \binom{n}{0}$.
- If $j = n$, it is $0 + 1 = 1 = \binom{n}{n}$.
- If $1 \le j \le n - 1$, put the terms over the common denominator $j!(n-j)!$:

$$\binom{n-1}{j} + \binom{n-1}{j-1} = \frac{(n-1)!(n-j)}{j!(n-j)!} + \frac{(n-1)!j}{j!(n-j)!} = (n-1)!\frac{n}{j!(n-j)!} = \binom{n}{j}.$$

Second identity. If $i < 1$ or $i > n$ both sides vanish: for $i = 0$ the left side has the factor $i = 0$ and $\binom{n-1}{-1} = 0$; for $i < 0$ and $i > n$ every coefficient is zero. If $1 \le i \le n$,

$$i\binom{n}{i} = i\frac{n!}{i!(n-i)!} = n\frac{(n-1)!}{(i-1)!(n-i)!} = n\binom{n-1}{i-1}.$$

□

Proof of Proposition 3. The space Π_n of real polynomials of degree at most n has dimension $n + 1$, with basis $1, t, \ldots, t^n$. The $n + 1$ polynomials $b_{0,n}, \ldots, b_{n,n}$ lie in Π_n, so it suffices to show that they are linearly independent. Let $\sum_{i=0}^{n} c_i b_{i,n}(t) = 0$ for all t. For $t \in (0, 1)$ divide by $(1-t)^n > 0$ and put $u = t/(1-t) \in (0, \infty)$:

$$\sum_{i=0}^{n} c_i \binom{n}{i} u^i = 0.$$

A polynomial in u with infinitely many roots is the zero polynomial, so $c_i\binom{n}{i} = 0$ for every i. Since $\binom{n}{i} \neq 0$, all $c_i = 0$. □

Proof of Proposition 4. For $t \in [0, 1]$ both t and $1 - t$ are non-negative, hence so is every $b_{i,n}(t)$. By the binomial theorem,

$$\sum_{i=0}^{n} \binom{n}{i} t^i (1-t)^{n-i} = (t + (1-t))^n = 1.$$

□

Proof of Proposition 6. With the convention $0^0 = 1$, $b_{i,n}(0) = \binom{n}{i}0^i$ equals 1 for $i = 0$ and 0 for $i \ge 1$. Likewise $b_{i,n}(1) = \binom{n}{i}0^{n-i}$ equals 1 for $i = n$ and 0 for $i < n$. Hence $B(0) = \sum_i b_{i,n}(0)P_i = P_0$ and $B(1) = \sum_i b_{i,n}(1)P_i = P_n$. □

Proof of Corollary 7. By Proposition 4, $B(t) = \sum_i \lambda_i P_i$ with $\lambda_i = b_{i,n}(t) \ge 0$ and $\sum_i \lambda_i = 1$. A convex combination of points lies in their convex hull by definition. □

Proof of Proposition 8. By linearity of M and Proposition 4, which gives $\sum_i b_{i,n}(t) = 1$,

$$\begin{aligned} A(B(t)) &= M\sum_i b_{i,n}(t)P_i + v \\ &= \sum_i b_{i,n}(t)MP_i + \left(\sum_i b_{i,n}(t)\right)v \\ &= \sum_i b_{i,n}(t)(MP_i + v) = \sum_i b_{i,n}(t)A(P_i). \end{aligned}$$

□

Proof of Theorem 10. By induction on r. For $r = 0$, $P_i^{(0)} = P_i = b_{0,0}(t)P_i$. Assume the claim holds for $r - 1$, for every admissible i. For $0 \le i \le n - r$ both i and $i + 1$ are admissible for $r - 1$, so

$$\begin{aligned} P_i^{(r)} &= (1-t)\sum_{j=0}^{r-1} b_{j,r-1}(t)P_{i+j} + t\sum_{j=0}^{r-1} b_{j,r-1}(t)P_{i+1+j} \\ &= \sum_{j=0}^{r}\Big[(1-t)b_{j,r-1}(t) + tb_{j-1,r-1}(t)\Big]P_{i+j}, \end{aligned}$$

where the second sum was re-indexed by $j+1 \to j$, and the convention $b_{-1,r-1} = b_{r,r-1} = 0$ covers the two ends. For $0 \le j \le r$ the bracket is

$$\binom{r-1}{j} t^j (1-t)^{r-j} + \binom{r-1}{j-1} t^j (1-t)^{r-j} = \binom{r}{j} t^j (1-t)^{r-j} = b_{j,r}(t)$$

by Lemma 2. For $r = n$ and $i = 0$ the claim reads $P_0^{(n)} = B(t)$. □

Proof of Lemma 11. Multiply $b_{i,n}(t)$ by $1 = (1-t) + t$:

$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n+1-i} + \binom{n}{i} t^{i+1} (1-t)^{n-i}.$$

Since $\binom{n+1}{i} = (n+1)/(n+1-i) \cdot \binom{n}{i}$ and $\binom{n+1}{i+1} = (n+1)/(i+1) \cdot \binom{n}{i}$ for $0 \le i \le n$,

$$\binom{n}{i} = \frac{n+1-i}{n+1}\binom{n+1}{i} = \frac{i+1}{n+1}\binom{n+1}{i+1}.$$

The first term is therefore $(n+1-i)/(n+1)$ times $b_{i,n+1}(t)$ and the second $(i+1)/(n+1)$ times $b_{i+1,n+1}(t)$. □

Proof of Proposition 12. By Lemma 11,

$$\begin{aligned} B(t) &= \sum_{i=0}^{n} \left[\frac{n+1-i}{n+1} b_{i,n+1}(t) + \frac{i+1}{n+1} b_{i+1,n+1}(t) \right] P_i \\ &= \sum_{k=0}^{n} \frac{n+1-k}{n+1} b_{k,n+1}(t) P_k + \sum_{k=1}^{n+1} \frac{k}{n+1} b_{k,n+1}(t) P_{k-1}, \end{aligned}$$

where $k = i$ in the first sum and $k = i+1$ in the second. The first sum may be extended to $k = n+1$, since its coefficient vanishes there, and the second to $k = 0$, for the same reason. Collecting the coefficient of $b_{k,n+1}(t)$ gives Q_k. □

A.2 Derivatives, reversal and subdivision

Proof of Lemma 13. We treat the possible values of i in turn.

- If $i < 0$ or $i > n$, then $b_{i,n} = 0$ and both terms on the right vanish.
- If $i = 0$, $b_{0,n}(t) = (1-t)^n$ has derivative $-n(1-t)^{n-1} = n\big[b_{-1,n-1}(t) - b_{0,n-1}(t)\big]$.
- If $i = n$, $b_{n,n}(t) = t^n$ has derivative $nt^{n-1} = n\big[b_{n-1,n-1}(t) - b_{n,n-1}(t)\big]$.
- If $1 \le i \le n-1$, the product rule gives

  $$b'_{i,n}(t) = \binom{n}{i}\big[it^{i-1}(1-t)^{n-i} - (n-i)t^i(1-t)^{n-i-1}\big].$$

  By Lemma 2, $i\binom{n}{i} = n\binom{n-1}{i-1}$, and, applying it to $n-i$ and using $\binom{n}{n-i} = \binom{n}{i}$, $(n-i)\binom{n}{i} = n\binom{n-1}{i}$. The two terms are therefore $nb_{i-1,n-1}(t)$ and $nb_{i,n-1}(t)$.

□

Proof of Theorem 14. By linearity and Lemma 13,

$$B'(t) = n\sum_{i=0}^{n} b_{i-1,n-1}(t) P_i - n\sum_{i=0}^{n} b_{i,n-1}(t) P_i.$$

In the first sum the term $i = 0$ vanishes; substituting $k = i-1$ gives $\sum_{k=0}^{n-1} b_{k,n-1}(t) P_{k+1}$. In the second the term $i = n$ vanishes. Hence

$$B'(t) = n\sum_{k=0}^{n-1} b_{k,n-1}(t)(P_{k+1} - P_k).$$

□

Proof of Corollary 15. By Theorem 14, B' is the Bézier curve of degree $n-1$ with control points $D_k = n(P_{k+1} - P_k)$, $k = 0, \ldots, n-1$. By Proposition 6 applied to it, $B'(0) = D_0$ and $B'(1) = D_{n-1}$. □

Proof of Lemma 16. For $0 \le i \le n$, since $\binom{n}{i} = \binom{n}{n-i}$,

$$b_{i,n}(1-t) = \binom{n}{i}(1-t)^i t^{n-i} = \binom{n}{n-i} t^{n-i}(1-t)^i = b_{n-i,n}(t).$$

For other i both sides are zero. □

Proof of Proposition 17. Re-index with $j = n - i$ and use Lemma 16:

$$\sum_{i=0}^{n} b_{i,n}(t) P_{n-i} = \sum_{j=0}^{n} b_{n-j,n}(t) P_j = \sum_{j=0}^{n} b_{j,n}(1-t) P_j = B(1-t).$$

□

Proof of Theorem 18. Left part. Since $1 - cs = (1-s) + s(1-c)$, the binomial theorem gives

$$\begin{aligned} B(cs) &= \sum_{i=0}^{n} \binom{n}{i} (cs)^i [(1-s) + s(1-c)]^{n-i} P_i \\ &= \sum_{i=0}^{n} \sum_{k=0}^{n-i} \binom{n}{i} \binom{n-i}{k} c^i (1-c)^k s^{i+k} (1-s)^{n-i-k} P_i. \end{aligned}$$

Substitute $j = i + k$, so that $0 \le i \le j \le n$ and $n - i - k = n - j$. Both $\binom{n}{i}\binom{n-i}{j-i}$ and $\binom{n}{j}\binom{j}{i}$ equal $n!/(i!(j-i)!(n-j)!)$, hence

$$\begin{aligned} B(cs) &= \sum_{j=0}^{n} \binom{n}{j} s^j (1-s)^{n-j} \sum_{i=0}^{j} \binom{j}{i} c^i (1-c)^{j-i} P_i \\ &= \sum_{j=0}^{n} b_{j,n}(s) P_0^{(j)}. \end{aligned}$$

The last step uses Theorem 10 with $r = j$ and $i = 0$: $P_0^{(j)} = \sum_{i=0}^{j} b_{i,j}(c) P_i$. This is the claim for the L_j.

Right part. Let $\overline{B}$ be the curve with control points $\overline{P}_i = P_{n-i}$, so that $\overline{B}(u) = B(1-u)$ by Proposition 17, and put $\overline{c} = 1 - c$. The left part applied to $\overline{B}$ gives

$$\overline{B}(\overline{c}s) = \sum_{j=0}^{n} b_{j,n}(s) \overline{L}_j,$$

where, by Lemma 16 and the substitution $l = j - i$,

$$\overline{L}_j = \sum_{i=0}^{j} b_{i,j}(1-c) P_{n-i} = \sum_{l=0}^{j} b_{l,j}(c) P_{n-j+l} = P_{n-j}^{(j)} = R_{n-j}.$$

The next-to-last equality is Theorem 10 with $r = j$ and $i = n - j$, an admissible index since $n - j \le n - r$. Therefore, using Lemma 16 once more,

$$\begin{aligned} B(c + (1-c)s) = \overline{B}(\overline{c}(1-s)) &= \sum_{j=0}^{n} b_{j,n}(1-s) R_{n-j} \\ &= \sum_{j=0}^{n} b_{n-j,n}(s) R_{n-j} = \sum_{k=0}^{n} b_{k,n}(s) R_k. \end{aligned}$$

□

Proof of Corollary 19. Write $a = t_0$ and $e = t_1$, so $0 \le a < e \le 1$. By Theorem 18 with $c = e$, the left part is the Bézier curve $u \mapsto B(eu)$ with control points $L_0, \ldots, L_n$. Apply Theorem 18 to it with $c' = a/e \in [0, 1)$ and keep the right part:

$$s \mapsto B\Big(e\Big(\frac{a}{e} + \Big(1 - \frac{a}{e}\Big)s\Big)\Big) = B(a + (e-a)s).$$

□

A.3 Cubic segments and paths

Proof of Proposition 21. Multiplying out the cubic Bernstein form gives

$$\begin{aligned} B(t) &= (1-t)^3P_0 + 3t(1-t)^2P_1 + 3t^2(1-t)P_2 + t^3P_3 \\ &= \big(1 - 3t + 3t^2 - t^3\big)P_0 + \big(3t - 6t^2 + 3t^3\big)P_1 + \big(3t^2 - 3t^3\big)P_2 + t^3P_3 \\ &= at^3 + bt^2 + ct + P_0, \end{aligned}$$

with a, b and c as stated, by collecting the powers of t.
Fix a coordinate k. The function B_k is a polynomial, hence continuous on the compact interval $[0,1]$, so it attains a minimum and a maximum there. Let t^* be a point where it attains one of them. If $t^* \in (0,1)$, then B_k is differentiable with an extremum at an interior point, so $B_k'(t^*) = 0$ (Fermat's theorem). Two cases remain.

- B_k' is not identically zero. It is a polynomial of degree at most 2, so it has at most two roots, and t^* is one of them if t^* lies in $(0,1)$. In every case $t^* \in T_k$.
- B_k' is identically zero. Then B_k is constant, so its extrema are attained at $t = 0$ as well, and $0 \in T_k$.

Therefore the minimum and the maximum of B_k over $[0,1]$ equal those over the finite set $T_k \subset [0,1]$, which are α_k and β_k as stated, since they are the infimum and supremum of B_k over the curve. By Definition 20, the bounding box is the product of the intervals $[\alpha_k, \beta_k]$: a box that contains the curve contains every value $B_k(t)$, hence α_k and β_k, and the product of these intervals contains the curve. □

Proof of Proposition 22. Line. The line $t \mapsto (1-t)P_0 + tP_3$ is a Bézier curve of degree 1 with controls $R_0 = P_0$ and $R_1 = P_3$. By Proposition 12 with $n = 1$, it is the quadratic with controls

$$Q_0 = P_0, \quad Q_1 = \frac{1}{2}P_0 + \frac{1}{2}P_3, \quad Q_2 = P_3.$$

Applying Proposition 12 again, with $n = 2$, gives the cubic with controls

$$\begin{aligned} S_0 &= Q_0 = P_0, \\ S_1 &= \frac{1}{3}Q_0 + \frac{2}{3}Q_1 = \frac{2}{3}P_0 + \frac{1}{3}P_3, \\ S_2 &= \frac{2}{3}Q_1 + \frac{1}{3}Q_2 = \frac{1}{3}P_0 + \frac{2}{3}P_3, \\ S_3 &= Q_2 = P_3, \end{aligned}$$

and $S_i = P_0 + \frac{i}{3}(P_3 - P_0)$ for $i = 0, \ldots, 3$.
Quadratic. By Proposition 12 with $n = 2$ and controls P_0, C, P_3, the cubic controls are

$$Q_0 = P_0, \quad Q_1 = \frac{1}{3}P_0 + \frac{2}{3}C, \quad Q_2 = \frac{2}{3}C + \frac{1}{3}P_3, \quad Q_3 = P_3,$$

and $Q_1 = P_0 + \frac{2}{3}(C - P_0)$, $Q_2 = P_3 + \frac{2}{3}(C - P_3)$.
Each step is an identity in t, so the curve and its parameterization are unchanged. □

Proof of Proposition 25. Write B_S and B_T for the cubic Bézier curves with controls P_i and Q_i, so that $S(u) = B_S((u - u_0 + h_S)/h_S)$ and $T(u) = B_T((u - u_0)/h_T)$. At $u = u_0$ the arguments are 1 and 0. By Proposition 6, $S(u_0) = P_3$ and $T(u_0) = Q_0$. By the chain rule and Corollary 15,

$$S'(u_0) = \frac{B_S'(1)}{h_S} = \frac{3(P_3 - P_2)}{h_S}, \quad T'(u_0) = \frac{B_T'(0)}{h_T} = \frac{3(Q_1 - Q_0)}{h_T}.$$

1. By Definition 23 with $k = 0$, C is C^0 at u_0 if and only if $S(u_0) = T(u_0)$, that is $P_3 = Q_0$.
2. With $P_3 = Q_0$, Definition 23 with $k = 1$ adds $S'(u_0) = T'(u_0)$, which is the stated equation after division by 3.
3. With $P_3 = Q_0$, by Definition 24 the join is G^1 if and only if $S'(u_0) \neq 0$, $T'(u_0) \neq 0$, and $T'(u_0) = \lambda S'(u_0)$ for some $\lambda > 0$. The two tangents are non-zero if and only if $P_3 \neq P_2$ and $Q_1 \neq Q_0$, and then

$$Q_1 - Q_0 = \left(\lambda \frac{h_T}{h_S}\right)(P_3 - P_2).$$

Conversely, if $Q_1 - Q_0 = \mu(P_3 - P_2)$ with $\mu > 0$, then $\lambda = \mu h_S / h_T > 0$ gives $T'(u_0) = \lambda S'(u_0)$.

4. A path of m segments under the uniform parameterization gives each segment the span $1/m$, so $h_S = h_T$ and the equation in 2 reduces to $P_3 - P_2 = Q_1 - Q_0$.

□

A.4 Constructions and interpolation

Proof of Proposition 26. For the curve B with the stated controls, Proposition 6 gives $B(0) = P_0$ and $B(1) = P_3$, and Corollary 15 gives

$$B'(0) = 3(P_1 - P_0) = D_0, \quad B'(1) = 3(P_3 - P_2) = D_1.$$

Now let Γ be any polynomial curve of degree at most 3 with $\Gamma(0) = P_0$, $\Gamma(1) = P_3$, $\Gamma'(0) = D_0$ and $\Gamma'(1) = D_1$. By Proposition 3, applied to each coordinate, there are unique $R_0, \ldots, R_3$ with $\Gamma(t) = \sum_i b_{i,3}(t) R_i$. The computation above, applied to the controls R_i, shows

$$\Gamma(0) = R_0, \quad \Gamma(1) = R_3, \quad \Gamma'(0) = 3(R_1 - R_0), \quad \Gamma'(1) = 3(R_3 - R_2).$$

Comparing with the four conditions gives $R_0 = P_0$, $R_3 = P_3$, $R_1 = P_0 + D_0/3$ and $R_2 = P_3 - D_1/3$. So R_i are the stated controls and $\Gamma = B$. □

Proof of Proposition 28. Put $G(s) = H(x_0 + hs)$. The map $H \mapsto G$ is a bijection of the polynomials of degree at most 3 onto themselves, with inverse $G \mapsto G((x - x_0)/h)$. By the chain rule, $H(x_i) = f_i$ and $H'(x_i) = m_i$ for $i = 0, 1$ are equivalent to

$$G(0) = f_0, \quad G(1) = f_1, \quad G'(0) = hm_0, \quad G'(1) = hm_1.$$

By Proposition 26 with $d = 1$, $P_0 = f_0$, $P_3 = f_1$, $D_0 = hm_0$ and $D_1 = hm_1$, exactly one polynomial G of degree at most 3 satisfies these four conditions, and its Bernstein coefficients are $c_0, \ldots, c_3$. Hence H exists, is unique, and equals $\sum_i b_{i,3}((x - x_0)/h) c_i$. □

Proof of Proposition 29. At $t = t_0$ and $t = t_1$ the segment is evaluated at $s = 0$ and $s = 1$, so $H(t_0) = P_0$ and $H(t_1) = P_3$ by Proposition 6. By the chain rule $H'(t) = B'(s)/h$. By Corollary 15, $B'(0) = 3(P_1 - P_0) = hD_0$ and $B'(1) = 3(P_3 - P_2) = hD_1$, so $H'(t_0) = D_0$ and $H'(t_1) = D_1$. Nothing requires $h > 0$. □

Proof of Proposition 30. Write the segment as $(X(s), Y(s))$. The controls of X are

$$x_0, \quad x_0 + \frac{h}{3}, \quad x_1 - \frac{h}{3} = x_0 + 2\frac{h}{3}, \quad x_1,$$

which by Proposition 22 (the line from x_0 to x_1) are those of the line $s \mapsto x_0 + hs$. Hence $X(s) = x_0 + hs$. The controls of Y are $y_0, y_0 + hm_0/3, y_1 - hm_1/3, y_1$, which are the coefficients $c_0, \ldots, c_3$ of Proposition 28 for the data $y_i = f(x_i)$ and $m_i = f'(x_i)$. Hence $Y(s) = H(x_0 + hs)$, and the segment is $s \mapsto (x_0 + hs, H(x_0 + hs))$. □

Proof of Theorem 31. The case $x \in \{x_0, x_1\}$ is trivial: both sides of the first formula vanish, for any ξ. Let $x_0 < x < x_1$, and put

$$E = f - H, \quad w(t) = (t - x_0)^2 (t - x_1)^2, \quad K = \frac{E(x)}{w(x)}, \quad g = E - Kw.$$

Then $w(x) > 0$, g is four times continuously differentiable, and $g(x_0) = g(x) = g(x_1) = 0$, because E and w vanish at x_0 and x_1, and $g(x) = E(x) - Kw(x) = 0$. Moreover $g'(x_0) = g'(x_1) = 0$: $E'(x_i) = f'(x_i) - H'(x_i) = 0$ by Definition 27, and w has a double root at each x_i, so $w'(x_i) = 0$.

By Rolle's theorem (Theorem 1.7 of Burden & Faires (2011)), g' vanishes at some $\xi_1 \in (x_0, x)$ and at some $\xi_2 \in (x, x_1)$. So g' has the four distinct zeros $x_0 < \xi_1 < \xi_2 < x_1$. The function g' is continuous and three times differentiable, so by the generalized Rolle theorem (Theorem 1.10 of Burden & Faires (2011)) there is a $\xi \in (x_0, x_1)$ with $g^{(4)}(\xi) = 0$.

Since H has degree at most 3 and w is a monic polynomial of degree 4, $H^{(4)} = 0$ and $w^{(4)} = 24$, so

$$0 = g^{(4)}(\xi) = f^{(4)}(\xi) - 24K, \quad \text{that is} \quad K = \frac{f^{(4)}(\xi)}{24}.$$

Hence $f(x) - H(x) = E(x) = Kw(x)$, which is the first formula.
For the bound, $(x - x_0)(x_1 - x)$ is a product of two non-negative numbers with the sum h, so it is at most $(h/2)^2$. Thus $w(x) \le h^4/16$ and

$$|f(x) - H(x)| \le \frac{M}{24} \cdot \frac{h^4}{16} = \frac{Mh^4}{384}.$$

□

A.5 Fitting and level sets

Proof of Corollary 32. Let $[a, b]$ be an interval at depth k, so $b - a = L/2^k$. On it, each coordinate of the Hermite segment is the cubic Hermite interpolant of the same coordinate of C (Proposition 29 and Proposition 28), so Theorem 31 bounds its error by $M(b - a)^4/384$. The Euclidean error at any parameter is at most $\sqrt{d}$ times the largest coordinate error, since $\|v\|_2 \le \sqrt{d}\|v\|_\infty$ for $v \in \mathbb{R}^d$: indeed $\|v\|_2^2 = \sum_k v_k^2 \le d\|v\|_\infty^2$. In particular the measured error, a maximum over probe parameters, satisfies

$$e \le \sqrt{d}M\frac{L^4}{384 \cdot 2^{4k}}.$$

This is at most ε if and only if $2^{4k} \ge \sqrt{d}ML^4/(384\varepsilon)$, that is, if $k \ge \frac{1}{4}\log_2\Big(\sqrt{d}ML^4/(384\varepsilon)\Big)$. Every depth $k \ge k^*$ satisfies this, so every interval at such a depth is accepted.
Suppose now that `max_depth` $\ge k^*$ and `max_segments` $\ge 2^{k^*}$. An interval is split only if it fails, which happens only at a depth $k < k^*$. So all intervals have depth at most k^* and no `ToleranceNotMet` is raised for the depth. The accepted intervals are disjoint pieces obtained by halving $[t_0, t_1]$ at most k^* times, so there are at most 2^{k^*} of them. A failing interval would give at least two accepted segments, so the number of segments accepted so far plus two never exceeds this final count, which is at most `max_segments`; the segment limit is therefore not exceeded either. □

Proof of Proposition 34. Let $\mathrm{rdp}(i, j)$ denote the recursive step of the algorithm on the indices $i < j$, which returns an increasing list of indices from i to j. We show by strong induction on $j - i$ that for every two consecutive indices $u < v$ in the returned list, every index l with $u \le l \le v$ satisfies $\mathrm{dist}(v_l, [v_u, v_v]) \le \varepsilon$.
Base case. If $j \le i + 1$, the list is (i, j), and the only indices between i and j inclusive are i and j, whose vertices are the chord's end points, at distance zero.
Step. Otherwise let δ^* be the largest distance from a vertex strictly between i and j to $\big[v_i, v_j\big]$. If $\delta^* \le \varepsilon$, the list is (i, j), and the claim holds by the choice of δ^* and because the end points have distance zero. If $\delta^* > \varepsilon$, the step chooses an index m with $i < m < j$ and returns the list of $\mathrm{rdp}(i, m)$ followed by that of $\mathrm{rdp}(m, j)$, with m occurring once. Two consecutive indices of the result are consecutive in one of the two lists. Since $m - i$ and $j - m$ are smaller than $j - i$, the induction hypothesis applies to both.
Steps 2 and 3 apply `rdp` between every two consecutive kept vertices of step 2 and concatenate the results. The output keeps exactly the returned indices $a_0 < \ldots < a_m$, and every two consecutive ones are consecutive in one of these lists. This proves the inequality. The `fit_error` of an output segment is $\max_{a_j \le i \le a_{j+1}} \mathrm{dist}\Big(v_i, \Big[v_{a_j}, v_{a_{j+1}}\Big]\Big)$, hence at most ε. □

Proof of Corollary 35.

1. The distance to a convex set is a convex function. For the segment

$K = [a, b]$, let x_1, x_2 be points with $\operatorname{dist}(x_i, K) \le \varepsilon$, attained at $y_i \in K$ (the minimum exists by compactness), and let $\lambda \in [0, 1]$. Then $\lambda y_1 + (1-\lambda) y_2 \in K$ and by the triangle inequality

$$\|\lambda x_1 + (1-\lambda)x_2 - (\lambda y_1 + (1-\lambda)y_2)\|_2 \le \lambda \|x_1 - y_1\|_2 + (1-\lambda)\|x_2 - y_2\|_2 \le \varepsilon.$$

So $\operatorname{dist}(\lambda x_1 + (1-\lambda)x_2, K) \le \varepsilon$.
A point x of the polyline lies on an edge $[v_i, v_{i+1}]$, so it is $\lambda v_i + (1-\lambda)v_{i+1}$. Choose j with $a_j \le i < a_{j+1}$; then $i + 1 \le a_{j+1}$, since indices are integers, and both v_i and v_{i+1} are within ε of $\left[v_{a_j}, v_{a_{j+1}}\right]$ by Proposition 34. By the convexity just proved, so is x.

2. A point left by step 1 is some v_i, and v_i lies within ε of

a chord by Proposition 34. A dropped input point p is within δ of the vertex kept before it, say v; then $\operatorname{dist}(p, [a, b]) \le \|p - v\|_2 + \operatorname{dist}(v, [a, b]) \le \delta + \varepsilon$ for a chord $[a, b]$ within ε of v. The final copy of the first point of a closed polyline is dropped under the same condition, with $v = v_0$. □

Proof of Proposition 40. Substituting the four corners of $[0, 1]^2$ into u shows that only one of the four terms is non-zero at each corner, with the coefficient 1, so u takes the corner values. At $s = t = 1/2$ each of the four products of weights equals $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$, so $u\left(\frac{1}{2}, \frac{1}{2}\right) = \frac{1}{4}(f_{00} + f_{10} + f_{11} + f_{01})$. □

Proof of Theorem 38. Since $\nabla F(p) \neq 0$, either $F_y(p) \neq 0$ or $F_x(p) \neq 0$.
Case $F_y(p) \neq 0$. Apply Theorem 37: there are I, J and a C^1 function $\varphi : I \to J$ with $\varphi(p_x) = p_y$ and $L_c \cap (I \times J) = \{(x, \varphi(x)) : x \in I\}$. Put $N = I \times J$ and $\gamma(x) = (x, \varphi(x))$. Differentiating $F(x, \varphi(x)) = c$ by the chain rule gives $F_x + F_y \varphi' = 0$ at $\gamma(x)$. By continuity of F_y, we may shrink I and J so that $F_y \neq 0$ on N; then

$$\gamma'(x) = (1, \varphi'(x)) = \left(1, -\frac{F_x}{F_y}\right) = \frac{1}{F_y}\left(F_y, -F_x\right),$$

which is non-zero and parallel to $\left(F_y, -F_x\right)$ at $\gamma(x)$.
Case $F_x(p) \neq 0$. Exchange the roles of x and y: there is a C^1 function ψ with $L_c \cap N = \{(\psi(y), y)\}$, curve $\gamma(y) = (\psi(y), y)$ and $\psi' = -F_y / F_x$, so

$$\gamma'(y) = \left(-\frac{F_y}{F_x}, 1\right) = -\frac{1}{F_x}\left(F_y, -F_x\right),$$

again non-zero and parallel to $\left(F_y, -F_x\right)$.
Finally $\nabla F \cdot \left(F_y, -F_x\right) = F_x F_y - F_y F_x = 0$. □

A.6 Export

Proof of Proposition 41. The difference of the curves is

$$\tilde{B}(t) - B(t) = \sum_{i=0}^{n} b_{i,n}(t)\left(\tilde{P}_i - P_i\right).$$

By Proposition 4, the weights $b_{i,n}(t)$ are non-negative and sum to 1, so for each coordinate

$$|\tilde{B}_k(t) - B_k(t)| \le \sum_{i=0}^{n} b_{i,n}(t)\, |\tilde{P}_{i,k} - P_{i,k}| \le \varepsilon.$$

Since $\|v\|_2 \le \sqrt{d}\|v\|_\infty$ for $v \in \mathbb{R}^d$ (Section 2.2 of Golub & Van Loan (2013)), we get $\left\|\tilde{B}(t) - B(t)\right\|_2 \le \sqrt{d}\varepsilon$.
The exporters write each coordinate with Python's fixed-point format with p decimals, which gives the multiple of 10^{-p} nearest to the coordinate; it differs from the coordinate by at most $\frac{1}{2} \cdot 10^{-p}$. The exporters also write a coordinate whose absolute value is less than $\frac{1}{2} \cdot 10^{-p}$ as 0, which changes it by that value, again at most $\frac{1}{2} \cdot 10^{-p}$. □

Change history

Release history of bezierkit. Documentation-only changes are not listed; each entry is tagged where the feature it describes is documented, following l3doc convention, so the page number is the real page.

v1.1.0 (2026-10-22)

General: Faster contour tracing, polyline fitting and path evaluation; the results are identical ··· 2

`trace_sampled`: Traces level sets of a field that is already sampled on a grid ····· 20

v1.0.0 (2026-10-04)

General: First stable release; the public API follows semantic versioning ····· 2

`bezierkit`: Continuous integration tests Python 3.10, 3.11, 3.12 and 3.13 ····· 3

`PiecewiseBezier.segment`: Returns the requested interval when `t0` falls inside a segment; previously the end point was wrong, because the path was split at `t0` and the suffix re-parameterized ····· 13

v0.5.0rc1 (2026-09-28)

General: First release on PyPI: degree-generic Bézier curves, cubic segments and paths, constructions, Hermite interpolation, fitting, level-set tracing, sampling, exporters and a command-line interface ····· 2

Command Index

A
approximate_path 24
B
BernsteinBasis 7
BernsteinEvaluator 8
BezierCurve 7
BezierCurve.derivative 9
BezierCurve.reversed 10
BezierCurve.segment 10
BezierCurve.split 10
C
construct_slopes 25
construct_tangents 25
ContourSet 22
CubicBezierSegment 11
CubicBezierSegment.bounding_box 12
D
DeCasteljauEvaluator 8
dumps 23
E
EndpointDerivatives 14
evaluate 25
F
fit_graph 17
fit_parametric 17
fit_polyline 18
from_path 24
from_svg_path_data 24
G
graph_hermite 15
H
hermite_error_bound 16
I
Interval 5
L
LevelContours 22
line_to_cubic 12
loads 23
M
maximum_polyline_deviation 19
P
ParameterValues 5
parametric_hermite 15
PathDocument 23
PiecewiseBezier 13
PlanarSlopes 14
Point 4
PointSet 5
Q
quadratic_to_cubic 12
S
Sample 22
sample 25
segment_to_tikz 24
T
TangentDirections 14
to_cubic 12
to_path 24
to_svg_path_data 24
to_tikz 24
trace_implicit 20
trace_sampled 20
U
UniformSampler 22
V
Vector 4

References

Apostol, T. M. (1974). *Mathematical Analysis* (2nd ed.). Addison-Wesley.

Athawale, T., & Johnson, C. R. (2019). Probabilistic Asymptotic Decider for Topological Ambiguity Resolution in Level-Set Extraction for Uncertain 2D Data. *IEEE Transactions on Visualization and Computer Graphics*, *25*(1), 1163–1172. https://doi.org/10.1109/TVCG.2018.2864505

Barsky, B. A., & DeRose, T. D. (1989). Geometric Continuity of Parametric Curves: Three Equivalent Characterizations. *IEEE Computer Graphics and Applications*, *9*(6), 60–69. https://doi.org/10.1109/38.41470

Bernstein, S. N. (1912). Démonstration du théorème de Weierstrass fondée sur le calcul des probabilités. *Communications De La Société Mathématique De Kharkov*, *13*, 1–2.

Bézier, P. (1966). Définition numérique des courbes et surfaces I. *Automatisme*, *11*, 625–632.

Burden, R. L., & Faires, J. D. (2011). *Numerical Analysis* (9th ed.). Brooks/Cole.

Douglas, D. H., & Peucker, T. K. (1973). Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or Its Caricature. *Cartographica*, *10*(2), 112–122. https://doi.org/10.3138/FM57-6770-U75U-7727

Farin, G. (2002). *Curves and Surfaces for CAGD: A Practical Guide* (5th ed.). Morgan Kaufmann.

Farouki, R. T. (2012). The Bernstein Polynomial Basis: A Centennial Retrospective. *Computer Aided Geometric Design*, *29*(6), 379–419. https://doi.org/10.1016/j.cagd.2012.03.001

Floater, M. S. (2025,). *An Introduction to Spline Theory*. https://www.uio.no/studier/emner/matnat/math/MAT4170/v25/undervisningsmateriale/spline_notes.pdf

Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.

Hershberger, J., & Snoeyink, J. (1994). An O(n \log n) Implementation of the Douglas–Peucker Algorithm for Line Simplification. *Proceedings of the Tenth Annual Symposium on Computational Geometry*, 383–384. https://doi.org/10.1145/177424.178097

Lorensen, W. E., & Cline, H. E. (1987). Marching Cubes: A High Resolution 3D Surface Construction Algorithm. *ACM SIGGRAPH Computer Graphics*, *21*(4), 163–169. https://doi.org/10.1145/37402.37422

Müller, A. (2024). A Tour d'Horizon of de Casteljau's Work. *Computer Aided Geometric Design*, *113*, 102366. https://doi.org/10.1016/j.cagd.2024.102366

Munkres, J. R. (1991). *Analysis on Manifolds*. Addison-Wesley.

Nielson, G. M., & Hamann, B. (1991). The Asymptotic Decider: Resolving the Ambiguity in Marching Cubes. *Proceedings of Visualization '91*, 83–91. https://doi.org/10.1109/VISUAL.1991.175782

Prautzsch, H., Boehm, W., & Paluszny, M. (2002). *Bézier and B-Spline Techniques*. Springer. https://doi.org/10.1007/978-3-662-04919-8

Ramer, U. (1972). An Iterative Procedure for the Polygonal Approximation of Plane Curves. *Computer Graphics and Image Processing*, *1*(3), 244–256. https://doi.org/10.1016/S0146-664X(72)80017-0

Rudin, W. (1976). *Principles of Mathematical Analysis* (3rd ed.). McGraw-Hill.

Stoer, J., & Bulirsch, R. (2002). *Introduction to Numerical Analysis* (3rd ed.). Springer. https://doi.org/10.1007/978-0-387-21738-3